Reporte de Práctica: Sumador de 16 bits estructural y despliegue hexadecimal en 7 segmentos

Mauricio Morán Riestra
Patricio Funes San Martín
Universidad de Las Americas Puebla (UDLAP)
Laboratorio de procesamiento de señales

11 de septiembre de 2026

Resumen

En esta práctica se diseña e implementa un sumador de 16 bits utilizando una descripción estructural basada en dos celdas: medio sumador (HA) y sumador completo (FA). El resultado de la suma se divide en nibbles para mostrarse en formato hexadecimal mediante cuatro decodificadores binario–a–7 segmentos, desplegándose en los cuatro displays de 7 segmentos (HEX0–HEX3) de la tarjeta Terasic DE10-Lite. La validación del diseño se realiza primero en simulación (Active-HDL) y posteriormente se compila e implementa en la FPGA usando Quartus Lite.

1. Objetivo

- Diseñar un sumador de 16 bits de forma estructural usando celdas HA y FA.
- Integrar el sumador con decodificadores binario a 7 segmentos para desplegar el resultado en hexadecimal.
- Simular el funcionamiento del sistema en **Active-HDL** (pruebas funcionales) y luego compilar/implementar en **Quartus Lite** para la DE10-Lite.

2. Estado del Arte

El desarrollo de sistemas de cómputo aritmético en mediante FPGAs se basa en la arquitectura de sumadores propagadores de acarreo. En el ámbito del diseño digital jerárquico, la implementación de sumadores de N bits se basa en la interconexión en cascada de celdas unitarias como Medios Sumadores (*Half Aadders*) y Sumadores Completos (*Full Adders*).

2.1. Sistemas de suma aritmética en FPGA

Diversos trabajos de diseño en HDL utilizan de estructuras en cascada para la ejecución de operaciones aritméticas básicas. Aunque existen arquitecturas avanzadas para acelerar la propagación del acarreo (tales como *Carry-Lookahead* o *Carry-Select*), la arquitectura *Ripple-Carry* ofrece una notable simplicidad en la descripción de la estructura jerárquica.

2.1.1. Medio Sumador (*Half Aadder*)

Un medio sumador suma dos bits a y b produciendo [1, 2]:

$$s = a \oplus b \tag{1}$$

$$c = a \cdot b \tag{2}$$

2.1.2. Sumador completo (*Full Aadder*)

Un sumador completo suma a, b y un acarreo de entrada cin [1, 2]:

$$s = (a \oplus b) \oplus C_{in} \tag{3}$$

$$C_{out} = (a \cdot b) + ((a \oplus b) \cdot C_{in}) \tag{4}$$

2.1.3. Sumador Ripple-Carry de 16 bits

Un sumador de 16 bits puede construirse conectando etapas en cascada mediante la propagación del acarreo, con un medio sumador en el bit menos significativo y quince sumadores completos para los bits restantes. A partir del segundo bit, cada etapa requiere una entrada de acarreo C_{in}.

- **Bit 0:** se implementa con un *HA*, donde cin = 0.
- **Bits 1 a 15:** se implementan con *FA*, cada uno recibiendo el acarreo de la etapa anterior.

2.2. Decodificador hexadecimal a 7 segmentos

El despliegue de datos binarios requiere de la decodificación de datos a 7 segmentos. Para ello, el resultado de 16 bits se divide en 4 nibbles:

$$SUM[15:0] = N3 = SUM[15:12], N2 = SUM[11:8], N1 = SUM[7:4], N0 = SUM[3:0]$$

Cada nibble se decodifica a los 7 segmentos (a,b,c,d,e,f,g) para mostrar un dígito hexadecimal (0–F) en un display. Es importante considerar que los arreglos de los displaysrequieren de una lógica negativa, ya que los displays (HEX0 a HEX5) integrados en el FPGA están activos en bajo (un segmento se enciende cuando el pin envía un 0 lógico y se apaga con un 1).

3. Planteamiento del Experimento

Para la ejecución de la práctica y las pruebas físicas del sistema se empleó el siguiente material:

- **Tarjeta de desarrollo FPGA:** Terasic DE10-Lite (basada en el dispositivo Intel MAX 10 10M50DAF484C7G).
- **Breadboard (Protoboard):** Tabla de pruebas de laboratorio para interconexión externa.
- **Cables de interconexión (*Jumpers*):** Cables tipo Macho-Hembra y Macho-Macho para la asignación de señales físicas desde los conectores de expansión GPIO de 3.3 V hacia la protoboard.
- **Diodo emisor de luz (LED):** Indicador luminoso externo para la señal de acarreo de salida (C_{out}).
- **Resistencia de limitación:** Resistencia de 330 Ω en serie con el LED externo para protección de corriente del pin GPIO.
- **Software de diseño y desarrollo:** Quartus Prime Lite Edition (síntesis y asignación de pines) y Active-HDL (simulación previa).

Para determinar lo lógica funcional del código, se decidió representarla a través de un diagrama a bloques, centrado en la distribución de operandos de entrada/salida.

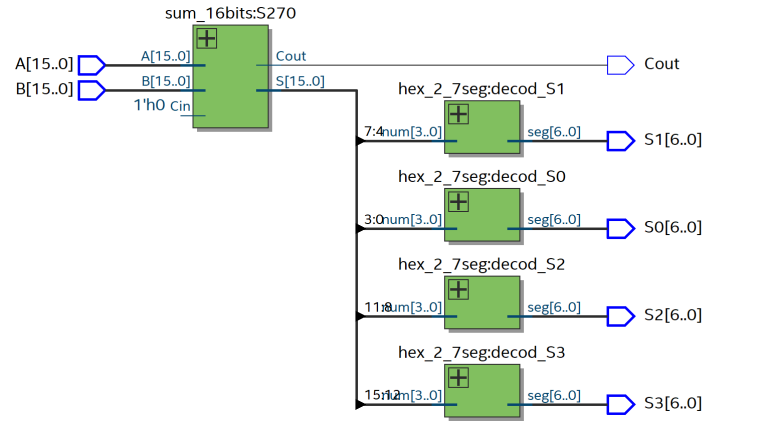


Figura 1: Esquemático de la lógica del programa.

El montaje experimental mostrado en la Figura 1 describe el funcionamiento del sumador y el decodificador. En términos más específicos, el diseño propuesto sigue la siguiente jerarquía:

1. **Half Adder**
2. **Full Adder**
3. **adder16:** conecta 1 HA y 15 FA en cascada (ripple-carry).

4. **Hex7seg:**decodificador combinacional de 4 bits a segmentos. top: integra entradas (switches), adder16 y 4 instancias de hex7seg para HEX0–HEX3.

En cuanto a otras especificaciones:

- Los 5 bits menos significativos del operando de cada operando (A y B) se asignaron a los 10 switches físicos integrados de la FPGA, operando con lógica *activo en alto*.
- Los 11 bits restantes de cada operando ($A[15:5]$ y $B[15:5]$) se direccionaron a los pines del cabezal de expansión GPIO de 2x20.
- Mediante cables *jumper*, dichos pines se conectaron a la línea de tierra (`GND`) de la protoboard para forzar las entradas a "0" lógico de forma predeterminada. La desconexión física del *jumper* permitió simular un estado "1" lógico mediante resistencias de *pull-up* internas del FPGA.

Para la visualización del resultado ($SUM[15:0]$ y C_{out}), se usaron los 4 displays de 7 segmentos de la tarjeta (`HEX0`–`HEX3`) operando en lógica *activa en bajo*. Por otro lado, el acarreo (C_{out}) se canalizó a través de un pin del puerto GPIO hacia la protoboard para encender el LED externo con su resistencia de protección de $330\,\Omega$.

4. Casos de Estudio o Pruebas

Para poner a prueba la FPGA, se definieron tres escenarios críticos de evaluación. El proceso de validación se dividió en dos etapas consecutivas: el modelado virtual adaptativo y la posterior experimentación en hardware.

4.1. Validación por Simulación

Antes de proceder con la síntesis física y el montaje en el laboratorio, el comportamiento del sistema jerárquico se validó de forma virtual. Los tres vectores de prueba representativos se introdujeron mediante un bloque de estímulos (*testbench*) dentro del entorno de desarrollo **Active-HDL**.

Las formas de onda obtenidas en el analizador lógico digital confirmaron la correcta sincronización funcional del diseño. La simulación demostró que la separación de los 16 bits del resultado en cuatro bloques independientes de 4 bits (*nibbles*) se ejecutaba de manera exacta. Asimismo, permitió verificar que las salidas destinadas a los displays realizaban la inversión lógica adecuada antes de que el archivo de programación fuera transferido a la FPGA, mitigando así el riesgo de fallas por asignación de rutas lógicas en el hardware real.

4.2. Pruebas en Hardware

Una vez comprobada la estabilidad del diseño en el simulador, se procedió a la carga del archivo de programación en la tarjeta DE10-Lite mediante **Quartus Lite**, aplicando los siguientes casos de estudio de manera controlada:

1. **Prueba de Suma Simple**
   - **Entrada A:** 0000 0000 0000 0001

- **Entrada B:** 0000 0000 0000 0001
- Los displays deben mostrar un número "2" en hexadecimal, con el led del acarreo apagado.

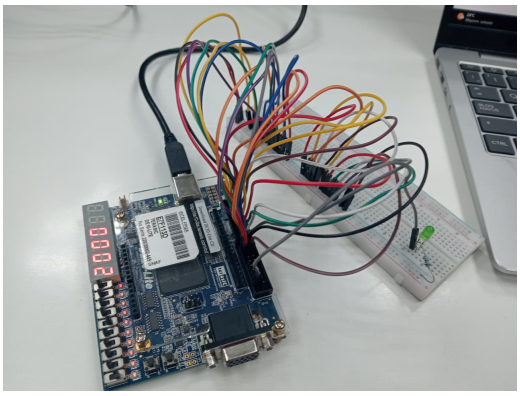


Figura 2: Primera prueba del FPGA.

2. **Prueba de Acarreos en Cadena**

- **Entrada A:** 1111 1111 1111 1111
- **Entrada B:** 0000 0000 0000 0001
- Lo que se debe ver es el led del acarreo encendido y los displays en "0000", esperando un resultado de "1000" en Hexadecimal.

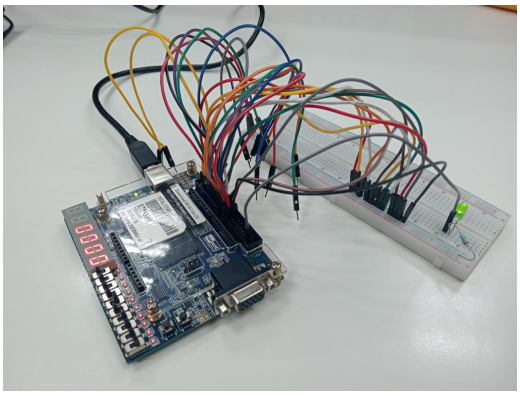


Figura 3: Segunda prueba del FPGA.

3. **Prueba de Capacidad Máxima**

- **Entrada A:** 1111 1111 1111 1111
- **Entrada B:** 1111 1111 1111 1111
- El led del acarreo debe estar encendido, seguido de la visualización de "$FFFE$" en los displays.

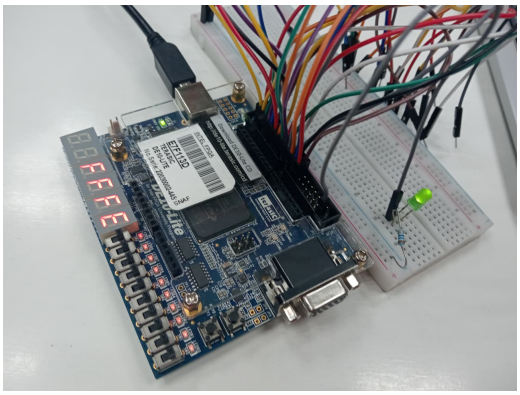


Figura 4: Tercera prueba del FPGA.

5. Análisis de Resultados

La validación funcional del sumador de 16 bits de forma estructural se llevó a cabo mediante tres casos de prueba críticamente seleccionados. La Tabla 1 sintetiza la comparación directa entre el cálculo aritmético teórico y las lecturas físicas desplegadas en la tarjeta de desarrollo. El comportamiento del circuito en cada escenario se analiza a continuación:

- **Caso de Prueba 1 (Suma Mínima):** Al ingresar los valores mínimos ($A = 0001$h y $B = 0001$h), el sistema respondió de manera inmediata mostrando `0002` en los displays de 7 segmentos y manteniendo el LED de acarreo apagado. Esta prueba inicial confirmó el correcto funcionamiento de la celda de Medio Sumador (*HA*) en el bit menos significativo (Bit 0) y la ruta de datos base sin propagación de acarreos hacia las etapas superiores.

- **Caso de Prueba 2 (Propagación del Acarreo en Cadena):** Al sumar el valor máximo representable en los registros ($A = $ FFFFh) con la unidad ($B = 0001$h), se forzó intencionalmente una transición crítica de acarreos. El resultado observado en los displays fue `0000` y el LED externo de C_{out} se encendió. Este comportamiento valida el funcionamiento de la arquitectura *Ripple-Carry*, demostrando que el bit de acarreo generado en el Bit 0 se propagó de forma secuencial y correcta a través de las 15 celdas de Sumador Completo (*FA*) hasta desbordar el sistema de manera exitosa.

- **Caso de Prueba 3 (Capacidad Máxima del Sistema):** Al realizar la suma de los valores límites máximos en ambos operandos ($A = $ FFFFh y $B = $ FFFFh), el sistema desplegó el valor numérico `FFFE` junto con el LED de acarreo encendido. Esta prueba de estrés confirma la estabilidad de las conexiones estructurales bajo las condiciones de mayor consumo de corriente y conmutación simultánea de compuertas lógicas dentro de la FPGA.

El análisis de estos tres escenarios confirma que tanto la propagación de acarreos en cascada como el bloque de decodificación binario-a-7 segmentos funcionan exactamente acorde al diseño esperado. Desde el punto de vista del hardware, se verificó experimentalmente la correcta implementación de la inversión lógica (lógica negativa) en el módulo `hex7seg`. Esto garantizó que los segmentos de los displays activos en bajo de la DE10-Lite

Prueba	Entrada A	Entrada B	Resultado Teórico	Displays (HEX)	LED C_{out}
1	0001h	0001h	0 0002h	0002	Apagado (0)
2	FFFFh	0001h	1 0000h	0000	Encendido (1)
3	FFFFh	FFFFh	1 FFFEh	FFFE	Encendido (1)

Tabla 1: Resumen de resultados teóricos vs. lectura en hardware DE10-Lite

encendieran con un '0' lógico y se apagaran con un '1' lógico, eliminando cualquier error de visualización o despliegue de caracteres erróneos.

6. Conclusiones

En esta práctica se implementó un diseño jerárquico y estructural que reforzó el uso de bloques básicos (HA y FA) para construir un sumador de 16 bits, además de integrar periféricos de salida (displays de 7 segmentos) mediante decodificación hexadecimal para mostrar los resultados. La validación realizada mediante la simulación previa a la síntesis permitió reducir errores de forma virtual, lo que facilitó significativamente la integración final en la FPGA.

Por último, el análisis de los casos de prueba extremos confirmó de manera empírica la viabilidad de la arquitectura de acarreo propagado (*Ripple-Carry*). La conmutación correcta del LED indicador ante escenarios de desbordamiento aritmético (C_{out}) y la estabilidad en el despliegue de datos bajo condiciones de máxima capacidad ($FFFE$h) demostraron que el sistema es robusto y tolerante al estrés digital. Esta experiencia fundamenta las bases necesarias para abordar en el futuro diseños aritméticos más complejos y optimizados, tales como sumadores por anticipación de acarreo (*Carry-Lookahead*) o unidades de procesamiento más avanzadas.

Referencias

[1] M. M. Mano y M. D. Ciletti, *Digital Design*, 6a ed. New York, NY, USA: Pearson, 2017.

[2] J. F. Wakerly, *Digital Design: Principles and Practices*, 4a ed. Upper Saddle River, NJ, USA: Pearson/Prentice Hall, 2006.

[3] *DE10-Lite User Manual*, Terasic Inc., Hsinchu, Taiwan, 2016.